Desenvolvimento orientado a testes

TDD: requisitos, testes e evolução do código



Prof. Dr. João Choma
Universidade Estadual de Maringá - DIN/CTC
github.com/JoaoChoma

Estratégias e técnicas de verificação, validação e teste de software

O que já estudamos?



- Casos de teste.
- Testes funcionais (caixa-preta), particionamento de equivalência e valores-limite.
- Testes estruturais (caixa-branca), grafo de fluxo de controle (GFC) e critérios de teste.
- Testes unitários com JUnit 5.
- Testes de integração, incluindo integração com Spring Boot.

Até aqui: aplicamos técnicas e critérios para selecionar testes e verificar código existente.

Objetivos de hoje



Ao final, você deverá ser capaz de:

1. Conhecer uma nova técnica de desenvolvimento: **TDD** (Test-Driven Development).
2. Explicar e executar o ciclo **RED, GREEN e REFACTOR**.
3. Distinguir teste aprovado, requisito atendido e código bem organizado.

Desenvolvimento convencional vs TDD



Convencional: testar após implementar

Requisito → implementação → teste → correções.

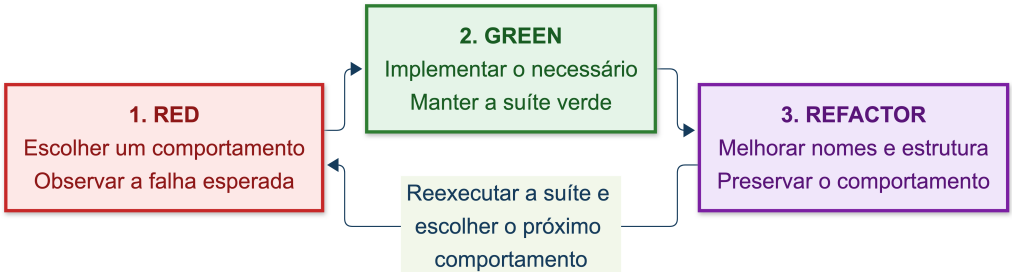
TDD: testar para orientar a implementação

Comportamento → teste que falha → implementação mínima
→ teste que passa → refatoração.

O que muda? O teste é escrito antes do código de produção e orienta o próximo passo do desenvolvimento.

As técnicas e os critérios de teste continuam valendo para orientar os testes. TDD é uma prática de desenvolvimento, não uma nova técnica de seleção de casos.

O ciclo RED, GREEN e REFACTOR



Por que observar a falha? Para verificar que o teste detecta a ausência do comportamento escolhido.

Erro de compilação pode iniciar o processo, mas ainda não é uma falha de asserção. Precisamos de estrutura compilável para observar o comportamento.

Exemplo: estacionamento com capacidade máxima



Objetivo criar um estacionamento com capacidade máxima de 10 veículos.

CT01 — Ocupação inicial
Dado de entrada: quantidades do tipo inteiro; Fluxo: criar um estacionamento; Resultado esperado: ocupação igual a **0**.

CT02 — Capacidade máxima
Dado de entrada: quantidades do tipo inteiro; Fluxo: criar um estacionamento com capacidade máxima de 10 veículos; Resultado esperado: capacidade máxima igual a **10**.

Estacionamento: duas regras, dois resultados



Regra	Comportamento esperado
RN01	Criar o estacionamento com uma capacidade máxima de veículos.
RN02	Um estacionamento recém-criado possui zero veículos estacionados.

Consideremos: 10 vagas vazias.
O que é **capacidade**? O que é **ocupação**?

Da regra ao caso de teste



	CT01 / RN02	CT02 / RN01
Objetivo	Verificar ocupação inicial	Verificar capacidade preservada
Entrada	Criar com capacidade 10	Criar com capacidade 10
Esperado	Ocupação igual a 0	Capacidade igual a 10

CT01 usa o construtor, mas sua asserção verifica apenas a ocupação.

Decisão de projeto: consultar a capacidade por meio de `getCapacidadeMaxima()`.

IntelliJ: escrever o teste antes da classe



```
@Test
void deveIniciarComZeroVeiculos() {
    Estacionamento estacionamento = new Estacionamento(10);
    assertEquals(0, estacionamento.getOcupacao());
}
```

Preparar: criar com capacidade 10.
Verificar: comparar a ocupação com o resultado esperado.

A classe ‘Estacionamento‘ ainda não existe.
Podemos usar a IDE para criar a classe, o construtor e o método.
Imports: `org.junit.jupiter.api.Test` e import estático de `Assertions.assertEquals`.

Compilar e testar comportamento



```java
public class Estacionamento {
    public Estacionamento(int i) { }

    public int getOcupacao() {
        throw new UnsupportedOperationException();
    }
}
```

RED: o teste executa, mas a consulta ainda não foi implementada.

GREEN: a solução é substituir a exceção por `return 0;` e executar.

Se a IDE gerar `return 0;` diretamente, CT01 já passa. Identificamos esse fato e seguimos para RN01, ainda não atendida.

RN01: observar uma falha de asserção



```java
@Test
void deveArmazenarCapacidadeMaxima() {
    Estacionamento estacionamento = new Estacionamento(10);
    assertEquals(10, estacionamento.getCapacidadeMaxima());
}
```

Vamos criar o método com retorno provisório 0 e execute a suíte.

Resultado esperado na demonstração

CT01 passa.
CT02 falha: esperado **10**, obtido **0**.

GREEN: preservar a capacidade informada



```
private int capacidadeMaxima;

public Estacionamento(int i) {
    this.capacidadeMaxima = i;
}

public int getCapacidadeMaxima() {
    return capacidadeMaxima;
}
```

Executar os dois testes: ambos devem passar.

Mas e as fronteiras? E se a capacidade informada fosse **25**?
Nsse caso precisamos distinguir o fato de armazenar o argumento de retornar sempre 10.

REFACTOR: melhorar sem criar um novo comportamento



```java
public class Estacionamento {
    private final int capacidadeMaxima;

    public Estacionamento(int capacidadeMaxima) {
        this.capacidadeMaxima = capacidadeMaxima;
    }
    public int getCapacidadeMaxima() {
        return capacidadeMaxima;
    }
    public int getOcupacao() { return 0; }
}
```

RN03: ocupação do estacionamento



RN03: uma entrada aumenta a ocupação.
Caso: ocupação 0, adicionar 10, resultado esperado 10.

```
@Test
void deveAdicionarUmVeiculo() {
    Estacionamento estacionamento = new Estacionamento(10);
    estacionamento.adicionarVeiculo();
    assertEquals(1, estacionamento.getOcupacao());
}
@Test
void deveAdicionarDoisVeiculos() {
    Estacionamento estacionamento = new Estacionamento(10);
    estacionamento.adicionarVeiculo();
    estacionamento.adicionarVeiculo();
    assertEquals(2, estacionamento.getOcupacao());
}
```

RED: método vazio mantém 0. **GREEN:**

```
this.ocupacao++;
```

RN04: impedir entrada quando lotado



RN04: O estacionamento não deve permitir a entrada de veículos quando atingir sua capacidade máxima.

Caso de teste CT04

- Capacidade máxima: 3 veículos.
- Pré-condição: estacionamento com 3 veículos.
- Ação: tentar adicionar mais um veículo.
- Resultado esperado: lançar uma exceção.
- Estado esperado: ocupação permanece em 3.

```
@Test
void naoDevePermitirEntradaQuandoLotado() {
    Estacionamento estacionamento = new
    Estacionamento(3);
    estacionamento.adicionarVeiculo();
    estacionamento.adicionarVeiculo();
    estacionamento.adicionarVeiculo();
    assertThrows(IllegalStateException.class, () ->
    estacionamento.adicionarVeiculo());
    assertEquals(3, estacionamento.getOcupacao());
}
```

Síntese: o que os testes permitem concluir



1. **Requisito e caso de teste:** a asserção precisa verificar o comportamento da regra. CT01 não substitui CT02.
2. **RED e GREEN:** observar a falha relevante e implementar o necessário, reexecutando os testes existentes.
3. **REFACTOR:** melhorar o código e verificar a preservação do comportamento com a suíte.

Limite: uma suíte verde dá evidências para os casos testados, não prova que todos os requisitos foram atendidos.

Próximo passo: especificar entrada e saída de veículos e seus limites antes de implementar essas operações.

Referências e estudo posterior



- SOMMERVILLE, Ian. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.
- JUNIT TEAM. *JUnit 5 User Guide*, versão 5.10.2.
  docs.junit.org/5.10.2/user-guide/

Ambiente da demonstração: Java 17, Maven, JUnit 5 e IntelliJ IDEA.
Estudo posterior: caderno complementar com os dois materiais, incluindo estoque, integração e refatoração.